



Seyed Mohammad Iman Mansouri

Game & Software Developer · AI Automation Systems Engineer · Unity Instructor

I build games and applications, I write the tools that build them, and I teach the same craft.

Mashhad, Iran

iman392392@gmail.com

+98 933 165 5416

Military service completed

6

games & apps built

2

titles published on
Iranian stores

315K

followers of a streamer
who played the game at
the event

17

live services in my own
system

84,000+

lines of code in the main
system

2+

years professional
studio experience

At a glance

I entered the game industry before finishing my diploma. Since 2023 I have worked at two game studios in Mashhad and contributed to shipping two mobile titles on Cafe Bazaar and Myket. Alongside studio work I built **Neon Run: Echo's Escape** – a 3D game that runs on mobile, PC and VR headsets, has six game modes, and was presented on a dedicated booth at the **Negative Five** accelerator event to visitors, streamers and studio directors.

Alongside products, I build tools. I built my own software-production system: 17 live services connecting code generation, image and audio generation, automated testing, long-term project memory and documentation – 84,290 lines of code across 432 files, with 385 automated tests.

Since 2025 I have also taught game programming with **Unity** and **C#** as a private, in-person instructor.

What I can do

in plain terms

1. Games from zero to release

Turning an idea into a playable game: gameplay code, character controller, camera, menus, save data, multiplayer modes, testing and store preparation – on mobile, PC and VR.

2. Applications and web apps

Interactive sites and apps with sign-up and login, cloud database, and an in-product AI assistant. Live example: the NitroLango language-learning platform.

3. Automation systems

Systems that carry out repetitive work on their own: code generation, image, audio and 3D-model generation, running tests, detecting errors and attempting to fix them.

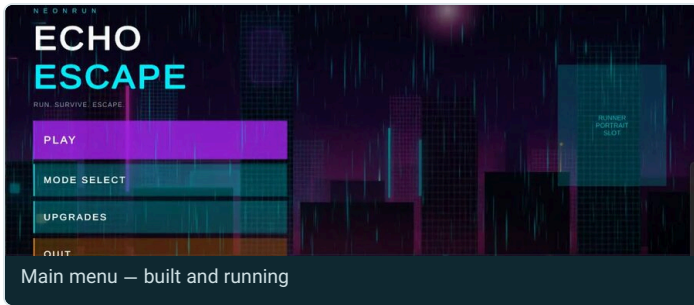
4. Teaching, documentation, knowledge transfer

I teach game programming privately, and turn a complex project into a precise manual so anyone else can continue the work and knowledge does not leave with one person.

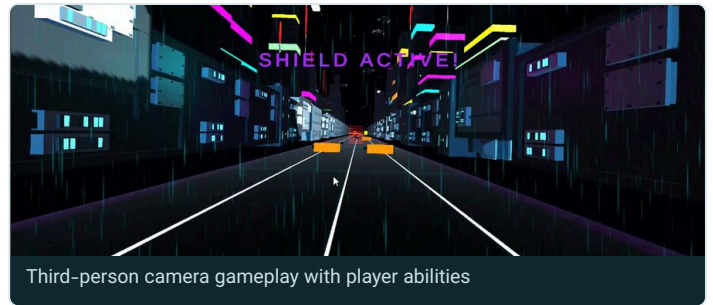
Neon Run: Echo's Escape

3D multi-platform game · main project

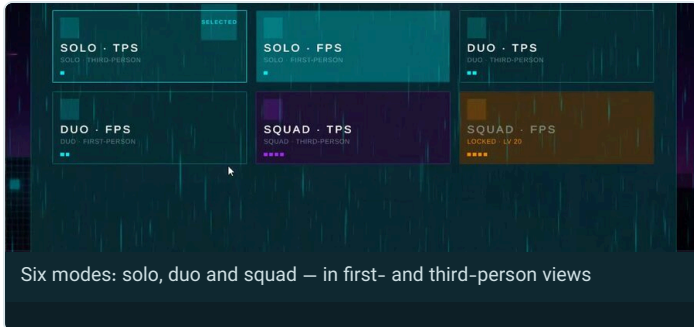
An endless runner set in a cyberpunk future: the year 2189, a city run by corporations, and a girl named Echo who has to escape the security system. The game has a narrative, five levels and two different endings. The technical build – code, camera, UI and multiplayer modes – is my work; game design was handled by my colleague.



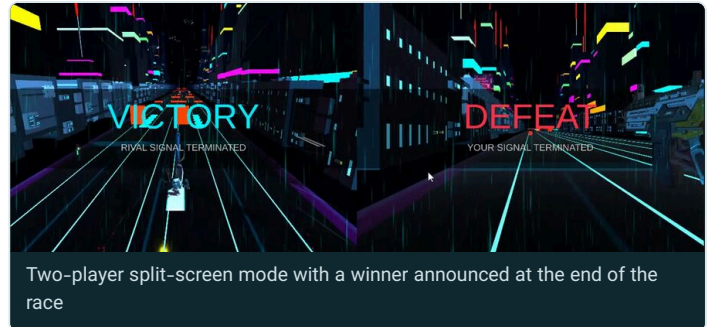
Main menu — built and running



Third-person camera gameplay with player abilities



Six modes: solo, duo and squad — in first- and third-person views



Two-player split-screen mode with a winner announced at the end of the race

What was built: character and camera control in two views, score and record system, special abilities (shield and boost), upgrade shop, infinite track and obstacle generation, competitive split-screen mode on a single device, menus and game-over screen, and identical behaviour across three platforms.

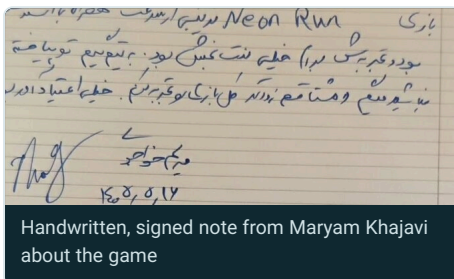
Status: in preparation for release. What was shown at the event was a demo build; the version in development has five levels, six modes and two endings.

Market feedback: who played the game

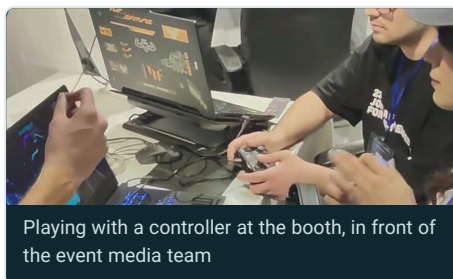
A playable build of Neon Run was presented on a dedicated booth at the Negative Five accelerator event, where dozens of visitors played it. Among them:

“... I really enjoyed the experience ... it was seriously addictive!”

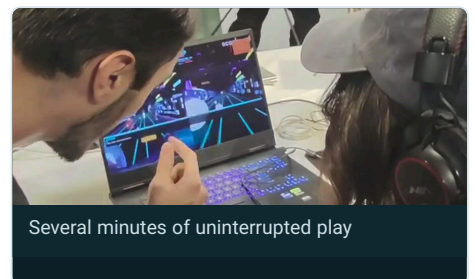
— Maryam Khajavi (iholymary), gaming content creator with over 315,000 Instagram followers and an active YouTube channel — after playing Neon Run at the event · handwritten and signed note



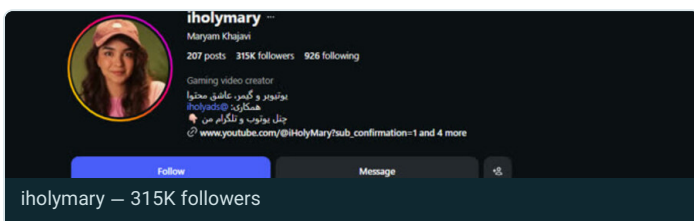
Handwritten, signed note from Maryam Khajavi about the game



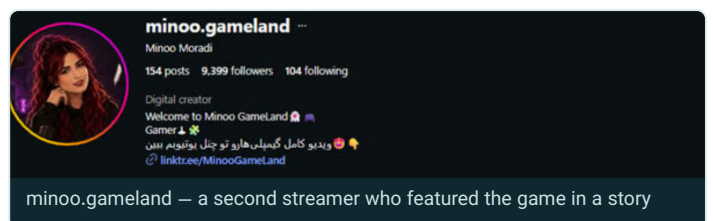
Playing with a controller at the booth, in front of the event media team



Several minutes of uninterrupted play

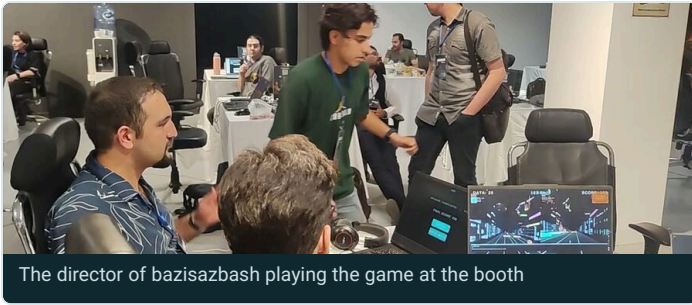


iholymary — 315K followers



minoo.gameland — a second streamer who featured the game in a story

The director of one of the oldest and largest game studios in Iran (basisazbash) sat down at the machine, played the game and gave positive feedback.



The director of bazisazbash playing the game at the booth



Gaming content creators presenting the game at the event



Maryam Khajavi — 315K

Instagram page of the content creator who played the game at the event



Minoo Gameland

Second streamer; story featuring the game from Negative Five



Bazisaz Bash

The studio whose director tried the game



NitroLango live build

Language-learning app — usable right now

UniversalSystem — software production system

systems engineering

UniversalSystem automates the repetitive stages of building software: code generation, image, audio and 3D-model generation, running tests, maintaining long-term project memory, and documentation. Like a production line where each stage runs on its own and only waits for a human on the decisions that matter. The output of a live health check is shown below.

84,290

lines of code across 432 files

17 / 17

live services healthy (verified)

385+

automated tests

What is inside the system?

each service, in plain terms

api-gateway entry point: every request arrives here first and is routed to the right part.

dotnet-api core logic: business rules, users and product data.

maf-orchestrator agent commander: splits work between several AI agents and tracks results.

celery-worker background worker: heavy, long-running jobs without blocking the user.

sqlserver main database: secure, structured data storage.

rabbitmq message queue: jobs wait in line and none are lost.

prometheus health watchdog: continuously collects metrics from every part.

jaeger tracer: shows each request step by step so slowdowns can be found.

web-frontend web interface: the screen people actually work with.

python-ai the AI brain: generates code, images, audio, music and 3D models.

node-microservices lightweight side services: real-time work and device communication.

matchmaker online match coordinator: pairs players for multiplayer sessions.

qdrant long-term memory: semantic search over everything the project has learned.

redis fast cache: instant answers for frequently used data.

mosquitto lightweight messaging for devices and sensors.

grafana visual dashboard: the state of the whole system at a glance.

proxy traffic routing between services.

vision-watcher the system's eye: looks at visual output and reports visual defects.

```

UniversalSystem :: live service verification
[ OK ] universal-system-api-gateway-1      Healthy
[ OK ] universal-system-api-gateway-2      Running
[ OK ] universal-system-python-es-1        Healthy
[ OK ] universal-system-python-es-2        Healthy
[ OK ] universal-system-dotnet-api-1       Healthy
[ OK ] universal-system-csharp-worker-1    Running
[ OK ] universal-system-scheduler-1        Healthy
[ OK ] universal-jagger                    Running
[ OK ] universal-system-frontend-1         Running
[ OK ] universal-protonthaus              Healthy
[ OK ] universal-ml-engineer              Healthy
[ OK ] universal-robotto                  Healthy
[ OK ] universal-system-frontend-2        Healthy
[ OK ] universal-robotto                  Healthy
[ OK ] universal-system-frontend-3        Healthy
[ OK ] universal-system-universal-proxy-1 Healthy
[ OK ] universal-robotto                  Healthy
[ OK ] universal-system-web-frontend-1     Running

== host-side HTTP probes (independent of Docker health status) ==
[ OK ] python-al      http://127.0.0.1:18000/health --> 200
[ OK ] dotnet-api     http://127.0.0.1:15000/health --> 200
[ OK ] api-gateway    http://127.0.0.1:15050/health --> 200
[ OK ] matchmaker     http://127.0.0.1:15001/health --> 200
[ OK ] robot-microservices http://127.0.0.1:18000/health --> 200
[ OK ] web-frontend   http://127.0.0.1:3000/health --> 200
[ OK ] protonthaus    http://127.0.0.1:15000/health --> 200
[ OK ] grafana        http://127.0.0.1:3001/pd/health --> 200
[ OK ] jagger         http://127.0.0.1:15688/health --> 200
[ OK ] elasticsearch  http://127.0.0.1:9200/_status --> 200
[ OK ] rabbitmq-rgnt  http://127.0.0.1:15672/health --> 200

== SUMMARY ==
services expected : 17
services verified : 17
services failed   : 0
HTTP probes OK    : 11/11

[ OK ] EVERY expected service verified up

```

Real output of a live system check: all 17 services up and healthy, and 11 independent health probes — all passing

Key capabilities of the system

- **Long-term project memory:** the system remembers what has been built and every decision taken, so it never starts from zero and does not break finished work.
- **Provider management:** connected to 11 different AI services. If one fails or runs out of quota, the system automatically moves to the next and work continues.
- **Safety layer:** dangerous commands that could delete project files are blocked — this part is covered by 22 separate tests.
- **Self-healing:** when an error occurs, the system reads it, finds the cause and attempts a fix instead of looping on the failure.
- **Automated content and scene building:** images, audio, music and 3D models generated from a text description, and automatic game-scene assembly — in one run the system placed over 176 objects across 30 sections inside the game engine.
- **Direct AI-assistant integration:** 9 of the system's tools are exposed directly to an assistant for use without an intermediary.
- **Installable elsewhere:** the whole body of knowledge is packaged as an add-on that has been installed and run on another machine.

Documentation and engineering discipline

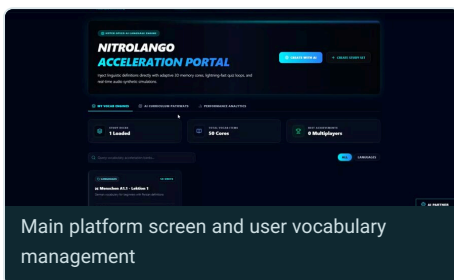
Every decision, every mistake and every solution in this project is recorded and documented. I built an internal knowledge library containing the getting-started guide, the open-task list, lessons learned from past failures, and operational runbooks — packaged as installable bundles.

A rule I set for myself: nothing is reported as “done” without evidence — file name, line number, and successful execution output.

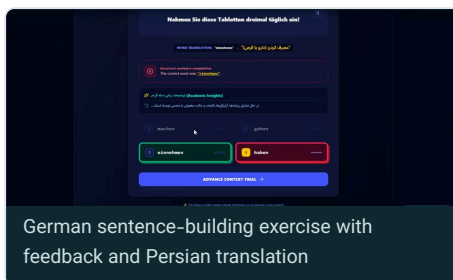
NitroLango — language-learning platform

AI-assisted web app · live build

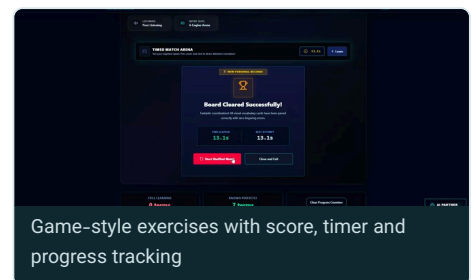
A complete language-learning platform I built from scratch: users sign up, their data is stored in the cloud, and exercises are designed as games rather than dry word lists — speed quizzes, listening practice, sentence building and a typing game. An in-product AI assistant generates exercises matched to the user's level.



Main platform screen and user vocabulary management



German sentence-building exercise with feedback and Persian translation



Game-style exercises with score, timer and progress tracking

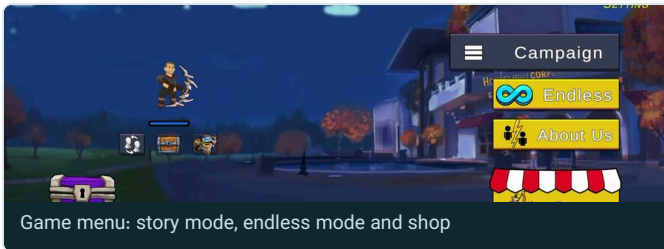
Live and usable: nitrolango-client.vercel.app · under continuous development.

Commercial games and studio projects

Arash Kamangir — commercial mobile game

My role: gameplay programmer and tools developer | Gametopia tech unit, Ferdowsi University of Mashhad Science & Technology Park
| 2024 – present

- Built the archery system: arrow trajectory calculation, collision, and real-time feedback to the player.
- Built a dedicated tool so the design team could lay out levels without writing code.
- Smoothed performance on low-end phones – reduced memory usage and eliminated frame hitches.



Parsa — published on Cafe Bazaar and Myket

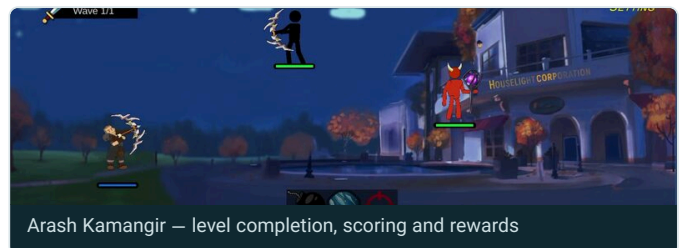
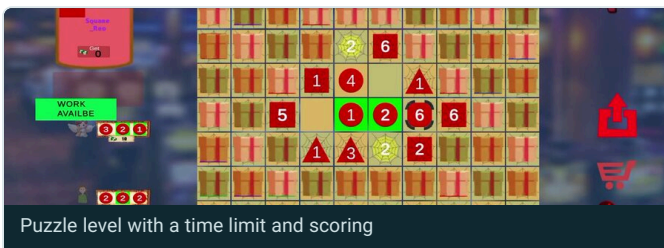
My role: technical and programming work only | Bomb Game Studio | 2023 – 2024 | one of my first two professional projects

- Programmed gameplay and UI sections through to official release on two established local stores.
- Player data persistence and runtime stability across different Android devices and versions.

Blood Castle / Design Villa — puzzle and interior-layout game

My role: logic and systems programmer | Bomb Game Studio | same genre as “Mina’s Villa” | my first professional project

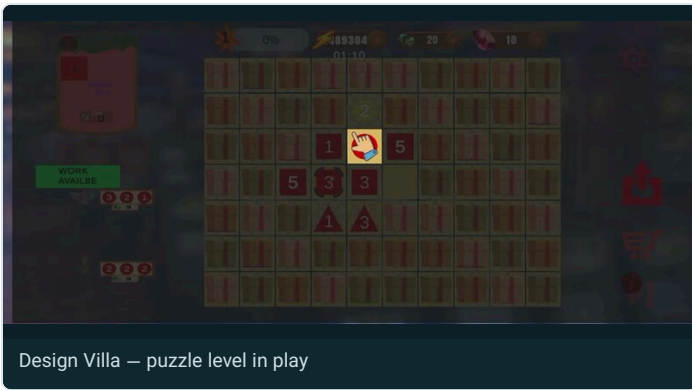
- Built the layout and decoration system so new items could be added without touching core code.
- Connected puzzle levels to story progression and reduced memory usage during image loading.



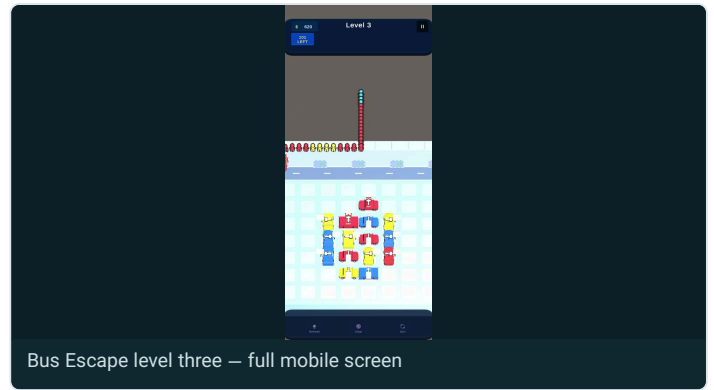
Bus Escape — traffic puzzle for mobile

My role: algorithm designer and game-logic developer | personal project

- Vehicle pathfinding on a grid: each vehicle must reach its own colour queue without getting stuck.
- Multi-layer level design with increasing difficulty, running smoothly on phones with minimal CPU cost.



Design Villa — puzzle level in play



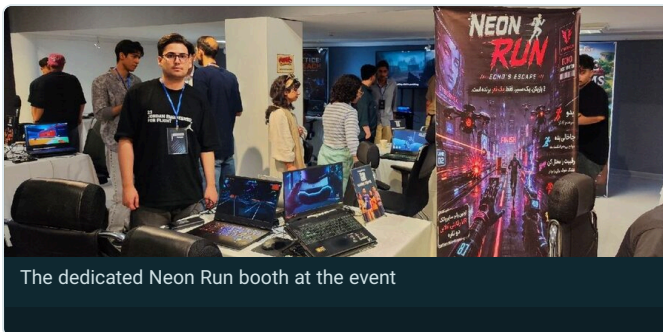
Bus Escape level three — full mobile screen

Events and presence in the game-development community

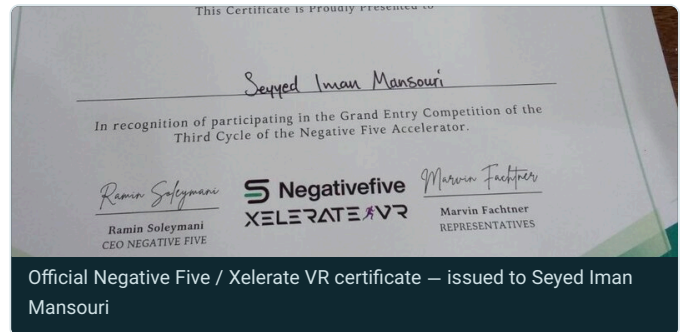
Negative Five Accelerator — third cycle · 2026

Entered Neon Run: Echo's Escape in the third-cycle entry competition; received an official certificate signed by the accelerator CEO and the Xelerate VR representative

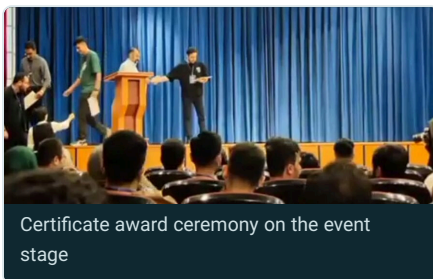
- Dedicated booth presenting a playable build, with direct player feedback collected on site.
- On stage at the event's certificate award ceremony.
- Media coverage of the game by streamers and gaming content creators.



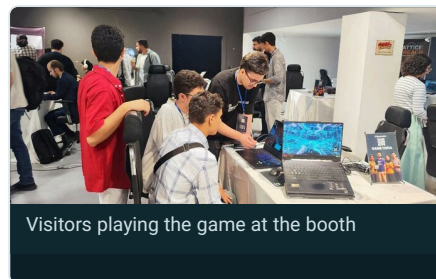
The dedicated Neon Run booth at the event



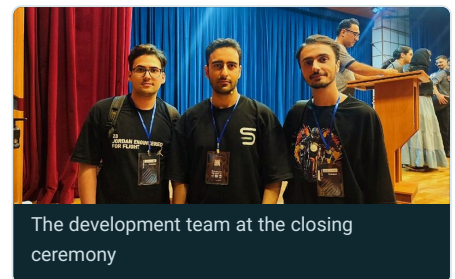
Official Negative Five / Xelerate VR certificate — issued to Seyed Iman Mansouri



Certificate award ceremony on the event stage



Visitors playing the game at the booth



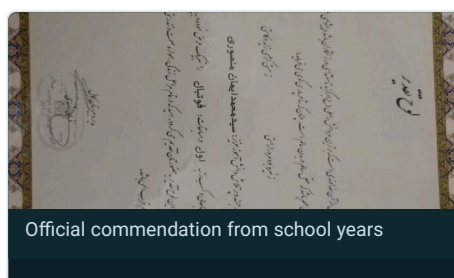
The development team at the closing ceremony

Before going professional

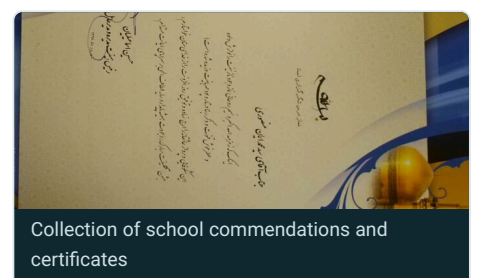
English course report cards from Shokouh Language Institute with scores of 91 to 100, plus school commendations. That same English is now the tool for reading technical documentation.



Distinguished English course report cards — scores 91 to 100



Official commendation from school years



Collection of school commendations and certificates

Teaching and knowledge transfer

private, in person · since 2025

My role: private instructor in game programming | Unity engine and C# | in-person sessions, Mashhad | 2025 – present

I run the course on the student's own project, not on prepared examples. The expected outcome: the student brings their own game to a final build on Android and Windows, and can read and fix its errors themselves.

Course syllabus I wrote

- **C# fundamentals:** variables, conditionals, loops, functions, classes and objects.
- **Working with Unity:** scenes, components, prefabs, physics and collisions.
- **Character and camera control:** movement, jumping, first- and third-person views.
- **UI and menus:** score system, records and player save data.
- **Project architecture:** layer separation, common patterns, code that stays changeable.
- **Debugging:** reading error messages, Debug, Profiler, and resolving complex failures.
- **Optimisation:** lower memory usage and smoother performance on weak phones.
- **Final build:** Android and Windows output, tested on a real device.

How a session runs

- The student writes the code; I guide, and we find the error together – instead of copying ready-made code.
- Each session adds one working feature to the student's project, not an abstract concept.
- Support continues between sessions for errors hit while practising.

Evidence available: the written course syllabus, recorded session videos, student project sources, and payment records – provided on request.

Video archive

19 videos · 56 minutes · online

Every item in this resume has recorded footage behind it. All 19 videos, each with a full write-up, sit on one web page – any browser, no install:

Neon Run: Echo's Escape – 6 videos

Solo first-person · solo third-person · two-player split-screen · full seven-minute session · abilities · endless run

Commercial games – 2 videos

Arash Kamangir · Bus Escape

Negative Five event – 9 videos

Maryam Khajavi (315K) at the booth · the bazisazbash studio head · the booth · certificate ceremony on stage · the venue

NitroLango – 2 videos

Full platform tour · latest update



Online video archive

albert392392.github.io/portfolio



Try the live product

Open the NitroLango app in your browser right now

Tools and technologies I work with

Unity (2D / 3D / VR)

C#

Python

.NET / ASP.NET Core

SQL Server

OpenXR / Meta Quest

Docker

Git

RESTful API

Redis

RabbitMQ

Vector Database (Qdrant)

Prometheus / Grafana

LLM Integration

Multi-Agent Automation

Model Context Protocol

Unity Profiler

Custom Editor Tools

Education and formal training

Technical & Vocational Diploma — Computer Networks and Software

June 2024

Shahed Isar Technical High School, Mashhad

Web Producer & Developer — official certificate

450 hours

Iran Technical & Vocational Training Organization

Game Dojo game-development bootcamp

full program

Building games in Unity, from project architecture to optimisation

Server-side programming with ASP.NET Core

400 hours

Building web services and application back-ends · in progress

SQL Server database design and administration

360 hours

Data structuring and query performance · in progress

Computer Networks — Network+

450 hours

Client-server infrastructure for online games · in progress

English courses since childhood

several terms

Shokouh Institute — distinguished report cards, scores 91 to 100

Languages

Persian	native	
English	B1+ — technical reading and writing	
German	A2 — improving	

Workstation

HP OMEN — personal development machine

Core i7-14700HX (20 cores) · 32 GB RAM · RTX 4060 · 1 TB SSD — used for VR testing and running AI models locally.

Current status and what is next

Neon Run: Echo's Escape

In preparation for release; the build in development has five levels, six modes and two endings.

NitroLango

Live build available and development continuing; next goal is fuller learning paths and a broader in-product assistant.

UniversalSystem

Shared infrastructure for the products that follow; completing services and adding wider test coverage.

Teaching and collaboration

Open to new private Unity students, and to work as a game and software developer or as part of a product team.



Try the live product

Open the NitroLango app in your browser right now

For a playable build, full gameplay video or a live product demo, get in touch:

iman392392@gmail.com · +98 933 165 5416